

BEGINNER-FRIENDLY BLUEPRINT

How the Finance Research Agent Works

From the moment you type a command, to the moment a tidy research report lands on your disk — explained so *anyone* can follow.

No jargon left unexplained

9 simple stages

Mapped to the real code

Think of it as a very careful research team: a librarian, a calculator robot, a panel of analysts, a quick debate, and a manager — working in a relay to answer one question, “*which stocks are worth a closer look?*”



Please read this first — straight from the project

This software produces **educational / research output only**. It is **not financial advice**, it places **no real orders**, and it uses **free public data** that may be stale or wrong. Always verify before acting.

Generated June 19, 2026 · a polished companion to `WORKFLOW.md` · built from HTML + CSS, printed to PDF via Chromium.

0 • The whole idea in 30 seconds

Imagine you ask a careful research team: “What are the best IT companies to invest in, in India?” Here's what happens behind the scenes:

- 1 A **librarian** figures out which companies you might mean and makes a list.
- 2 A **calculator robot** downloads free public numbers (profits, debt, price trends) and scores each company — pure math, no opinions — plus a “how trustworthy is this data?” sticker.
- 3 A **panel of AI analysts** reads those scores and writes short, evidence-based opinions. They *cannot* make up numbers.
- 4 **Two AIs debate** — one optimistic “Bull,” one cautious “Bear.”
- 5 A **risk & tax checker** adds rules specific to you (e.g. local capital-gains tax, or “keep it low-risk for a student”).
- 6 A **manager** combines everything into a final ranked list and writes you a tidy report.



The one analogy to remember

The robot does the **maths**; the AI only does the **talking**. Numbers always come from real data and formulas — the AI is never allowed to invent a figure. That's what makes the output more trustworthy than asking a chatbot to “just pick stocks.”

1 • What this project actually is

The **Finance Research Agent (FRA)** is a command-line program that ranks the “best” stocks to *research* for a theme (like “best IT stocks in India”) or a single company. It blends **deterministic financial math** (reproducible, never guesses) with **AI language models** (which explain and add context). The result is a written research report tailored to one of two saved **profiles**: an Indian adult investor, or a German student investor.



In plain English: “deterministic”

Deterministic just means “the same inputs always give the same answer — no randomness, no guessing.” Run it twice with the same data and you get identical numbers. FRA even prints a short fingerprint (an `input_hash`) so you can prove it.

2 • Visual glossary

Every heavy word in this document is defined here once, in plain language, with a quick everyday comparison. Skim it now, or flip back whenever a term trips you up.



CLI (command-line interface)

A program you drive by typing a command instead of clicking buttons.

Like texting an order to the kitchen instead of pointing at a menu.



Ticker

A stock's short code, e.g. `INFY.NS` (Infosys) or `SAP.DE` (SAP).

Like a phone contact's short name.



Universe

The full pool of candidate companies before the list is narrowed down.

The whole bookshelf, before you pick a few books.



Profile

A saved set of preferences (country, currency, tax, risk appetite).

Your saved coffee order the barista already knows.



Factor

A measurable quality of a company. FRA uses five: Quality, Value, Momentum, Financial Health, Earnings Quality.

Like grading a car on safety, mileage, price, looks and reliability.



Composite score

One number from 0 to 1 that blends the five factors using your chosen weights. Higher ranks better.

A single star-rating made from several sub-scores.



Percentile rank

"Better than X% of the others" — scored against peers, not an absolute bar.

A class rank, not a fixed exam pass-mark.



LLM (large language model)

An AI that reads and writes text (GPT, Claude, or a local Llama). Here it plays "analyst."

A very well-read intern who can summarise, but mustn't invent figures.



Data health card

A traffic-light (OK / WARN / CRITICAL) showing how complete and trustworthy this run's data was.

The freshness sticker on a carton of milk.



Coverage

How much of the expected data actually showed up. Low coverage → the score is nudged toward neutral.

A half-filled form: you trust its conclusions less.



Snapshot

A tidy bundle of one company's numbers (price, margins, debt, momentum...).

A nutrition label for a stock.



Bull vs Bear

Two AIs argue the optimistic (Bull) vs cautious (Bear) case so you see both sides.

A friendly courtroom: prosecution and defence.



In plain English: "agent," "pipeline," "orchestrator"

An **agent** is just one worker that does a single job. A **pipeline** is the conveyor belt those workers stand along. The **orchestrator** is the floor manager who runs them in the right order and passes the work down the line.

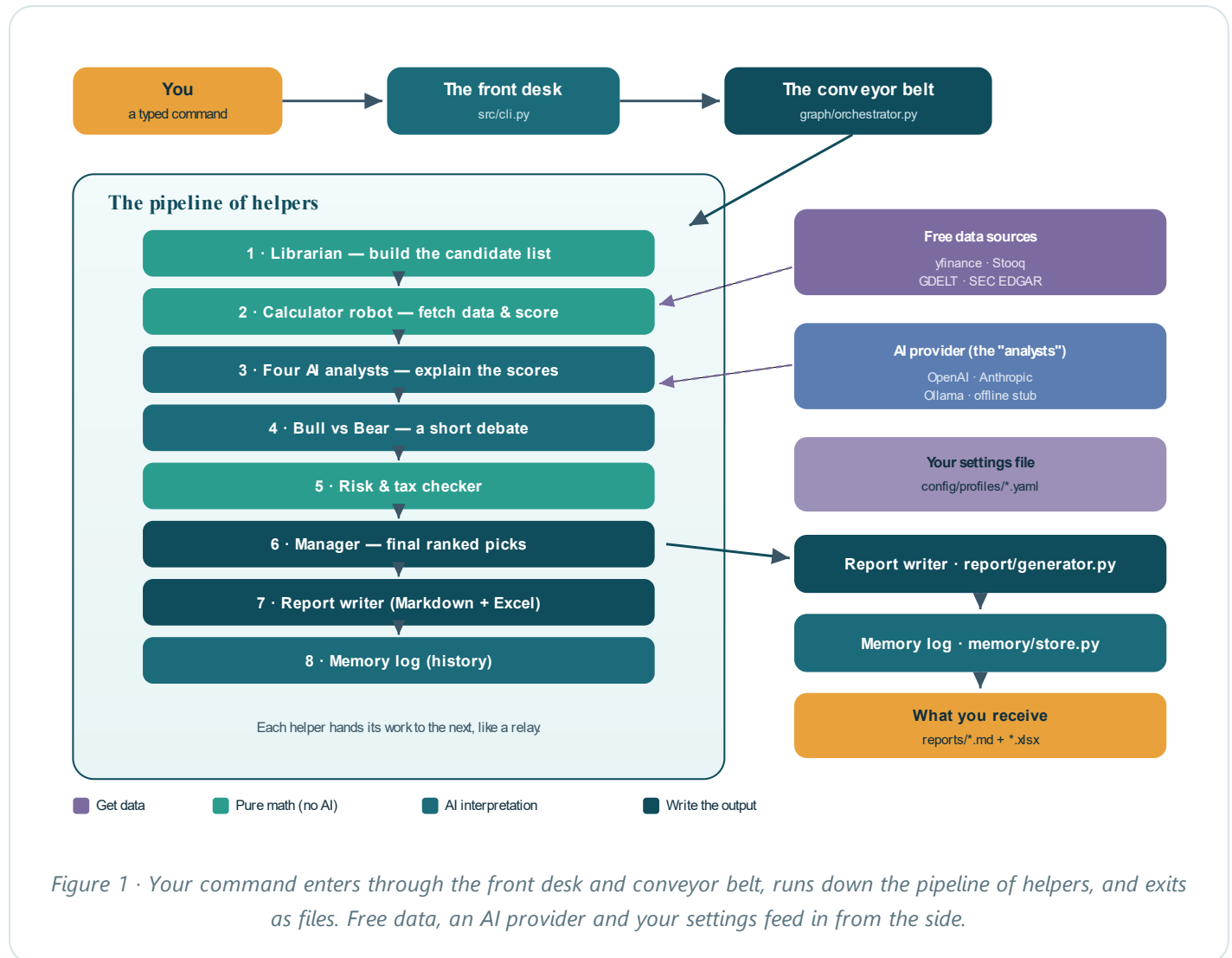


A note on "MCP"

You may see **MCP (Model Context Protocol)** mentioned in the code — a standard way for AI tools to call outside data. It is *not* wired into FRA today; the shipped version talks to free sources (yfinance / Stooq) directly. It's noted only as a possible future swap.

3 · How the pieces fit together

FRA is a **chain of helpers**. Your typed question enters on the left and flows through each stage, getting richer at every step, until it becomes a report. A single shared “clipboard” (called the **AgentState**) travels the whole way, collecting results.



Three layers, in plain terms

1 · Get data

Fetch numbers & news from free sources and cache them on disk.
(src/data/)

2 · Think

Deterministic math scores companies; AI interprets and debates. (src/factors/, src/agents/)

3 · Write it down

Produce the Markdown / Excel report and log the run.
(src/report/, src/memory/)



The whole thing in one line (from the README)

Universe → DataProvider → FactorEngine → Analysts → Bull/Bear Debate → Risk+Profile → Manager → Report

4 • The 9 stages, start to finish

The orchestrator runs the stages below in order. After the calculator robot finishes, there's one fork in the road — if you asked for no AI (or nothing made the shortlist), it takes a shortcut straight to the manager.

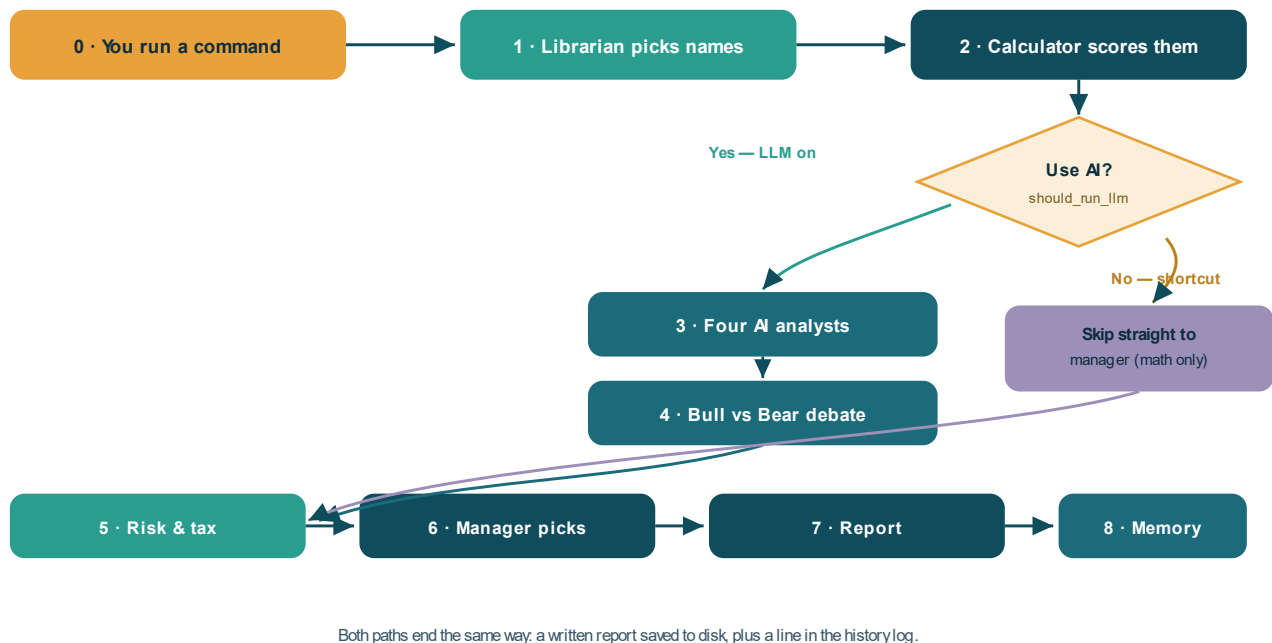


Figure 2 • The stage-by-stage flow, including the one decision point that chooses between the full AI path and the fast math-only shortcut.



The one fork in the road: “Use AI?”

Right after Stage 2, FRA asks `should_run_llm`. If you passed `--no-llm`, or nothing made the shortlist, it **skips Stages 3–4** and jumps straight to the manager using math only — then risk, report, memory. You still get a complete report.



You run a command YOU

You type a request in the terminal — which profile to use, what to research, how many picks you want.

Handled by `research()` in `src/cli.py`; it packs everything into a shared `AgentState`.

1

The librarian builds the candidate list NO AI

It works out whether you typed one company or a theme ("best banks in India"), spots the sector from keywords, and pulls the matching companies. Live lists first, built-in lists as backup. **No AI — just rules.**

`src/agents/universe.py` → produces `candidate_tickers`.

2

The calculator robot fetches data & scores everyone NO AI

The heart of the whole thing. It downloads a **snapshot** for each company (cross-checking the price against a second free source), builds the **data health card**, then scores every company on the five factors and blends them into one **composite score**. Companies with thin data get pulled toward neutral so a lucky gap can't fake a high score.

`src/agents/quant.py` + the factor engine in `src/factors/engine.py` → `shortlist`, `data_health`, a reproducible `input_hash`.

3

Four AI analysts explain the numbers AI

Now the AI joins — but on a tight leash. Four analysts (**fundamentals**, **technical**, **news**, **macro**) read the scores and write short, plain opinions. Each is **forbidden from inventing numbers** and must cite the metric behind every claim. No AI available? Each one falls back to a simple math rule.

`src/agents/fundamentals.py`, `technical.py`, `news_sentiment.py`, `macro.py` → `analyst_signals`.

4

Bull vs Bear hold a short debate AI

Two AI personas — an optimist and a sceptic — argue the 2–3 most interesting names, so the report shows both the upside and the worries.

`src/agents/researchers.py` → `debate` turns (repeat with `--rounds`).

5

The risk & tax checker adds your rules NO AI

Applies *your* profile rules: a cap on how much any single name can take, volatility filters, ETF preference, and tax notes (Indian capital-gains, or German Abgeltungssteuer) copied straight from your settings file.

`src/agents/risk_profile.py` → `risk_notes`, `tax_notes`.

6

The manager makes the final ranked picks MIXED

Combines the scores, the analyst notes and the debate into a final ranked list. Each pick gets a 2–3 sentence reason (citing real metrics), key risks, a suggested holding time and an estimated **after-tax** return. Forgotten a shortlisted name? It's quietly backfilled.

```
src/agents/manager.py ( run / run_quant_only ) → picks .
```

7

The report writer saves your document OUTPUT

Turns everything into a clean Markdown report (and, unless you opt out, a multi-sheet Excel workbook) saved into `reports/`.

```
src/report/generator.py → report_path (+ excel_path ).
```

8

The memory log remembers the run OUTPUT

Appends a compact record of the run so the `history` command can list it later.

```
src/memory/store.py → a line in .fra_memory/index.jsonl .
```


5 • Inputs & outputs

What you provide

Option	What it means
<code>--profile</code> (required)	<code>india_adult</code> or <code>germany_student</code>
<code>--target</code> (required)	a ticker (<code>SAP.DE</code>) or a theme ("best IT stocks in India")
<code>--top</code>	how many picks to return (default 10)
<code>--universe</code> / <code>--domain</code>	narrow the candidate pool
<code>--rounds</code>	number of Bull-vs-Bear debate rounds (default 1)
<code>--no-llm</code>	deterministic, AI-free run
<code>--no-excel</code>	skip the Excel workbook
<code>--as-of</code> YYYY-MM-DD	evaluate as of a past date

What you get back

- A Markdown report in `reports/` and (optionally) an **Excel workbook**.
- A **console summary**: a "Top picks" table and a data-health line.
- A **history entry** you can later list with `python -m src.cli history`.



What's inside the report (a real example)

A header (target, universe, top-N, fingerprint hash) and the disclaimer; a **data health card** (e.g. *OK, 6/6 tickers fetched, 82% coverage*); factor warnings; the **final picks** (each with score, coverage, confidence, suggested horizon, after-tax estimate, a short thesis, risks and tax notes); a factor-breakdown table; the analyst signals; the Bull-vs-Bear transcript; risk & tax notes; and a methodology section.

6 • Configuration & setup

Install & run (Windows / PowerShell)

```
# set up a clean environment and install dependencies
python -m venv .venv && .venv\Scripts\activate
pip install -r requirements.txt
copy .env.example .env

# (optional) a local, free AI:
ollama pull llama3.1:8b
ollama serve

# run a research pass:
python -m src.cli research --profile india_adult --target "best IT stocks in India" --

# offline / deterministic (no AI):
python -m src.cli research --profile india_adult --target "best banks in India" --top
```

Settings that live in `.env`

Variable	Purpose	Default
LLM_PROVIDER	openai, anthropic, ollama, or cursor_io	ollama
LLM_MODEL	provider-specific model name	llama3.1:8b
LLM_TEMPERATURE	randomness (0 = deterministic)	0
OPENAI_API_KEY / ANTHROPIC_API_KEY	keys for hosted AIs	empty
OLLAMA_HOST	local Ollama endpoint	http://localhost:11434
FRA_CACHE_DIR	where data is cached	./.fra_cache



No API key? It still runs.

If no AI is reachable, FRA quietly swaps in a deterministic **stub** and every analyst falls back to its math rule. You still get a valid, factor-based report — just with sparser AI prose.



In plain English: “config-as-data”

The two profile files in `config/profiles/` hold the factor weights, tax rules, risk limits and currency. So most behaviour can be changed by editing a **settings file**, with no Python edits:

`india_adult.yaml` (INR, NIFTY, momentum-tilted) and `germany_student.yaml` (EUR, DAX/MDAX, lower-risk).

7 • A worked example

Running `research --profile india_adult --target "best IT companies in India" --top 10 :`

- 1

Front desk loads the India profile, builds the clipboard, prints a banner.
- 2

Librarian spots “IT” → Information Technology, and lists TCS, Infosys, Tech Mahindra, Wipro, HCLTech, LTIMindtree.
- 3

Calculator fetches each snapshot (data health: *OK, 6/6 fetched, 82% coverage*) and scores them; fingerprint `1aceed39273729da` .
- 4

Analysts weigh in — e.g. TCS looks strong on fundamentals (ROE 48.4%) but weak on momentum (−31.9%); every claim cites a real number.
- 5

Debate: Bull says “world-class franchises after a reset”; Bear says “the cycle is still rolling over.”
- 6

Risk + tax adds a 10% per-name cap and Indian capital-gains notes.
- 7

Manager ranks them (TCS #1, Infosys #2...), each with a thesis and after-tax estimate. One name had 0% data, so it's flagged low-confidence — graceful degradation in action.
- 8

Report + memory save the file and log the run.

8 • Want to change something?

You want to...	Edit this
Add a new investor profile	Add a YAML in <code>config/profiles/</code> (copy an existing one)
Change weights / tax / risk limits	Edit the relevant <code>config/profiles/*.yaml</code> — no code change
Add or change a factor	Add an extractor in <code>src/factors/metrics.py</code>
Swap the data source	Re-implement <code>DataProvider</code> in <code>src/data/provider.py</code>
Add a new analyst	New module in <code>src/agents/</code> , then wire it into the orchestrator
Change the report layout	Edit <code>src/report/templates/report.md.j2</code>
Use a different / no AI	Set <code>LLM_PROVIDER</code> , or pass <code>--no-llm</code>

File map: `src/cli.py` (entry) · `src/graph/orchestrator.py` (pipeline) · `src/graph/state.py` (shared clipboard) · `src/agents/*` (workers) · `src/factors/*` (math) · `src/data/*` (data + caching) · `src/report/*` (output) · `src/memory/store.py` (history) · `config/profiles/*.yaml` (settings).